

HTML & CSS & JAVASCRIPT

Initiation à l'HTML & CSS & JAVASCRIPT

Préparé par Herizo Randrianarivonjy

HTML5

1. Présentation HTML 5

2. La syntaxe

- a. C'est quoi W3C, balise HTML:
- b. Attributs dans les balises
- c. Imbrications des balises
- d. Éléments vides (images...)
- e. Structure d'une page html

3. Environnement de développement (Visual Code Studio)

4. Encodage

5. Les balises:

- a. Contenu du <Body>
- b. Les liens hypertextes
- c. Les images
- d. Les tableaux
- e. Les éléments de division de contenu
- f. Menu de navigation
- g. Les formulaires
- h. Données multimédia: intégration vidéo, google map, musique audio
- i. L'inspecteur

6. Quelques balises à savoir:

CSS

1. Présentation

2. Structure d'un fichier css

- a. Structure
- b. C'est quoi un sélecteur?
 - i. Types de sélecteurs:

3. Mettre du css dans une page HTML

4. Outil de débogage dans un navigateur

5. Modèle de boîte en css

- a. Les boîtes en bloc:
- b. Les boîtes en ligne:
- c. Les composants d'une boîte
- d. Les modèles de boîtes
- e. Les marges

	f. Les bordures
	g. Le padding
6.	Les polices d'écritures
	a. Comment utiliser une police d'écriture classique?
	b. Importer une police d'écriture en locale
	c. Importer une police d'écriture distante
7.	Les couleurs et unités de mesures
8.	Positionnement
	a. Pixel ou pourcentage
	b. Position
9.	Responsive design
	a. Média queries
	b. Images responsives
	i. Image responsive avec un width à 100%
	ii. Commutation de résolution: tailles différentes
	iii. Commutation de résolution: même taille, résolutions différentes
	iv. Le méta-tag du viewport
10.	L'héritage
	a. Qu'est ce que l'héritage en css?
	b. Comment fonctionne le reset css?
11.	Présentation de bootstrap
12.	Exemple d'utilisation bootstrap
JAVASCRIPT	
1.	Présentation de JAVASCRIPT
2.	Types et grammaire
	Les bases du langage
	Commentaires
	Déclarations
	Structure de données et types
3.	Instructions conditionnelles, boucles et itérations
	Instruction if...else
	Switch
	Gestion des erreurs et des exceptions
	Boucles et itérations:
	L'instruction for:
	L'instruction do...while:
	L'instruction while:
	L'instruction break:
	L'instruction continue:
	L'instruction for...in:
	L'instruction for...of:
4.	Les fonctions
5.	Les classes
6.	Gestion d'évènement en JAVASCRIPT
7.	JS Promise

- [a\) Callbacks](#)
- [b\) Promise object](#)
- [c\) JS async/await](#)

[8. JS Modules](#)

[Export:](#)

[Import:](#)

[9. JS HTML DOM](#)

- [a. DOM Introduction](#)
- [b. DOM Methods](#)
- [c. DOM Document](#)
- [d. DOM css](#)
- [e. DOM nodes](#)

[10. JS WEB API](#)

- [a. Web API introduction](#)
- [b. Web Storage API](#)
- [c. Web fetch API](#)

HTML5

1. Présentation HTML 5

HTML: Hyper Text Markup Language.

HTML à été créé par Mr Tim Berners-Lee vers 1991 mais publié en 1995 avec la version HTML 2.0, la dernière version c'est la 5.0.

Définition: L'HTML C'est un langage de balisage qui permet de créer des pages web. Elle permet aussi de décrire la structure d'une page web.

2. La syntaxe

a. C'est quoi W3C, balise HTML:

Le W3C c'est l'organisation responsable de la standardisation et de l'évolution du langage HTML. C'est elle qui établit et dicte les normes d'interprétation que les navigateurs doivent suivre. Par conséquent, l'affichage d'un même code doit être la même dans tous les navigateurs.

W3C met à disposition un espace de validation disponible sur le lien suivant <https://validator.w3.org/>

N.B: l'importance de cette validation est que notre page web soit compatible avec la plupart des navigateurs.

b. Attributs dans les balises

Une balise c'est un mot entre chevron par exemple:

```
<mabalise></mabalise>
```

Certaines balises sont auto fermantes:

```
</mabalise>
```

Un attribut dans une balise permet de fournir des informations supplémentaires à la balise en question, par exemple:

```
<mabalise id="id-unique"></mabalise>
```

c. Imbrications des balises

Il est possible d'imbriquer des balises, comme pour les parenthèses en mathématiques $()()$.

```
<p>  
  <span>  
    </span>  
</p>
```

d. Éléments vides (images,...)

Les éléments vides en HTML ne peuvent pas avoir des éléments imbriqués. Les éléments vides n'ont qu'une balise ouvrante.

Comme exemple nous pouvons citer les balises suivantes:

1. `<img>` qui permet d'afficher une image;
2. `<hr>` qui permet d'afficher une ligne;
3. `<input>` qui permet par exemple de créer une zone de texte, un checkbox, bouton radio;
4.

Il y en a d'autres mais nous ne listerons que cela... vous pouvez consulter le site https://developer.mozilla.org/fr/docs/Glossary/Void_element pour avoir une liste plus complète.

e. Structure d'une page html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Page Title</title>  
    <meta charset="utf-8">  
    <meta name="keywords" content="motclé1 motclé2">  
    <meta name="description" content="la description du site web">  
    <meta name="author" content="Nom Prenom">  
  </head>  
  <body>  
  
    <h1>My First Heading</h1>  
    <p>My first paragraph.</p>  
  
  </body>  
</html>
```

C'est quoi l'*HTML DOCTYPE*?

HTML doctype fait référence à la DTD(Document Type Definition). DTD fait référence à un document qui représente tous les éléments autorisés dans une page web.

`<html>`: C'est l'élément principale qui permet d'englober le code de votre page web.

`<head>`: Cet élément permet de définir les entêtes de votre page web. Par exemple, la balise `<title>` est imbriqué dans `<head>` afin de spécifier un titre pour votre page web.

`<title>`: Permet de donner un titre à une page web.

`<link>`: Permet de lier une feuille de style à une page.

`<meta>`: Permet de fournir des détails à propos d'une page web. Elle est très importante si on veut que notre site web soit bien référencé et facilement trouvable par google et autre moteur de recherche.

3. Environnement de développement (Visual Code Studio)

Installer une version de visual code studio compatible avec votre pc

Utiliser live server pour tester votre html en temps réel.

4. Encodage

```
<meta charset="utf-8">
```

Ce genre de balise nous permettra d'afficher ou d'utiliser des caractères issus de presque toutes les langues humaines de notre document HTML.

ASCII (American Standard Code for Information Interchange) un norme de codage de caractère qui ne gère que 128 caractères différents entre autres: les chiffres 0-9, les lettres anglaises A-Z et certains caractères spéciaux comme ! \$ + - () @ < > .

ISO-8859-1 le jeu de caractères par défaut de HTML 4 mais ne prenait en charge que 256 codes de caractères différents.

ANSI (Windows-1252) qui est le jeu de caractères windows d'origine est identique à *ISO-8859-1* mais gère 32 caractères supplémentaires.

UTF-8 gère presque tous les caractères et symboles du monde.

5. Les balises:

a. Contenu du <Body>

La partie visible de votre page HTML sera entre la balise:

```
<body>  
    Contenu HTML  
</body>
```

b. Les liens hypertextes

Les liens dans une page HTML sont des hyperliens car ils permettent de passer d'un document à un autre en cliquant dessus. Quand on survole à l'aide d'une souris un lien, le curseur de votre souris se transformera en un petit main.

Syntax:

```
<a href="url">link text</a>
```

L'attribut href permet d'indiquer le lien vers l'élément HTML ou bien une autre page web. Par contre, *link text* est la partie que le visiteur de la page web verra.

The target attribute:

Par défaut, quand on clique sur un lien, la page de destination s'affiche dans l'onglet courant du navigateur.

Avec l'attribut target on peut spécifier où est ce qu'on veut ouvrir le document mentionné dans l'attribut href du lien.

L'attribut target peut avoir les valeurs suivantes:

- `_self`: Default. Ouvre le document dans la même fenêtre ou onglet;
- `_blank`: Ouvre le document dans une nouvelle fenêtre ou onglet;
- `_parent`: Ouvre le document dans la frame "parent" ou dans le même onglet;
- `_top`: Ouvre le document dans l'onglet, une fenêtre ou un `<iframe>` le plus haut sinon se comporte comme `_self`.

Utiliser un image comme un lien vers une autre page web

```
<a href="default.html">  
    
</a>
```

Utiliser un lien pour ouvrir un programme d'envoi d'email sur le pc de l'utilisateur

```
<a href="mailto:someone@example.com">Send email</a>
```

Utiliser un bouton comme un lien mais pour cela il faut ajouter un peu de javascript

```
<button onclick="document.location='default.asp'">HTML Tutorial</button>
```

c. Les images

La balise est un élément vide donc ne possède pas de balise fermante. Les attributs suivants doivent être renseigner:

- src: Permet de spécifier le chemin de l'image.
- alt: Permet de renseigner un texte pour l'image. Le texte sera affiché quand le navigateur ne trouvera pas l'image.

```

```

On peut ajouter les attributs width et height pour renseigner la dimension de l'image.

```

```


d. Les tableaux

Voici un simple tableau en HTML

```
<table>
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
  </tr>
  <tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
  </tr>
</table>
```

Chaque ligne du tableau doit commencer par <tr> et terminer par </tr>.

Les titres des colonnes du tableau sont renseignés dans les balises <th></th>.

Voici quelques tags utiles dans les tableaux:

<thead>: Permet de regrouper les entêtes du tableau.

<tbody>: Permet de regrouper le contenu du corps du tableau.

<colgroup>: Spécifie un groupe pour une ou plusieurs colonne du tableau.

```
<table>
  <colgroup>
    <col span="2" style="background-color:red">
    <col style="background-color:yellow">
  </colgroup>
  <tr>
    <th>ISBN</th>
    <th>Title</th>
    <th>Price</th>
  </tr>
  <tr>
    <td>3476896</td>
    <td>My first HTML</td>
    <td>$53</td>
  </tr>
</table>
```

e. Les éléments de division de contenu

`<div>`: ou division, un élément HTML qui n'a aucun effet sur le contenu tant qu'une mise en forme à l'aide du css n'est pas appliquée. Cet élément fait partie de la catégorie de contenu appelé "*Contenu de flux*".

Les éléments appartenant à cette catégorie contiennent généralement du texte ou du contenu intégré.

Ex: `<a>`, `<h1>`, `<h2>`,....

f. Menu de navigation

On peut fabriquer des menus de navigation à partir des liens `<a>`, pour que nos visiteurs puissent naviguer dans notre page web.

```
<nav>
  <a href="/html/">HTML</a> |
  <a href="/css/">CSS</a> |
  <a href="/js/">JavaScript</a> |
  <a href="/python/">Python</a>
</nav>
```

Il est possible d'avoir des menus à l'horizontal ou vertical selon nos besoins. Pour avoir ce genre d'orientation de menu il faut appliquer du css sur les éléments de notre menu.

g. Les formulaires

Le langage HTML permet de construire des formulaires qui peuvent être mis en forme par du css. Voici un exemple de code HTML qui permet d'avoir un formulaire sur une page web

```
<form>
  .
  form elements
  .
</form>
```

Un formulaire HTML permet de collecter les données entrées par les visiteurs d'une page web. La balise `<form>` est utilisée pour créer un formulaire HTML.

```
<form action="/action_page.php" method="get">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname">
  <button type="submit" id="btnValider">Valider</button>
</form>
```

L'attribut *action* permet de définir l'action qui doit être faite quand le formulaire sera soumis par l'utilisateur en cliquant sur le bouton Valider.

L'attribut *method* permet de définir la méthode de requête HTTP à utiliser lors de la soumission du formulaire.

Avec la méthode GET:

- Les données du formulaire seront ajoutées à l'url ou l'action du formulaire.
- Ne jamais utiliser GET pour soumettre des données sensibles.
- La longueur de l'url est limitée à 2048 caractères.

Par contre avec la méthode POST:

- Ajoute les données du formulaire au body de la requête HTTP.
- Il n'y a pas de limite, on peut soumettre d'important volume de données.

h. Données multimédia: intégration vidéo, google map, musique audio

`<video>`:

```
<video width="320" height="240" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

L'attribut *controls* ajoute les boutons play, pause et volume.

L'élément `<source>` permet de spécifier plusieurs sources de vidéo. Le navigateur utilisera la première qui sera compatible.

<audio>:

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
Your browser does not support the audio element.
</audio>
```

Google map: afin d'afficher une carte google map sur notre page web il est nécessaire d'utiliser du javascript.

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Google Map</h1>

<div id="googleMap" style="width:100%;height:400px;"></div>

<script>
function myMap() {
var mapProp= {
  center:new google.maps.LatLng(51.508742,-0.120850),
  zoom:5,
};
var map = new
google.maps.Map(document.getElementById("googleMap"),mapProp);
}
</script>

<script
src="https://maps.googleapis.com/maps/api/js?key=YOUR_KEY&callback=myMap"></script>

</body>
</html>
```

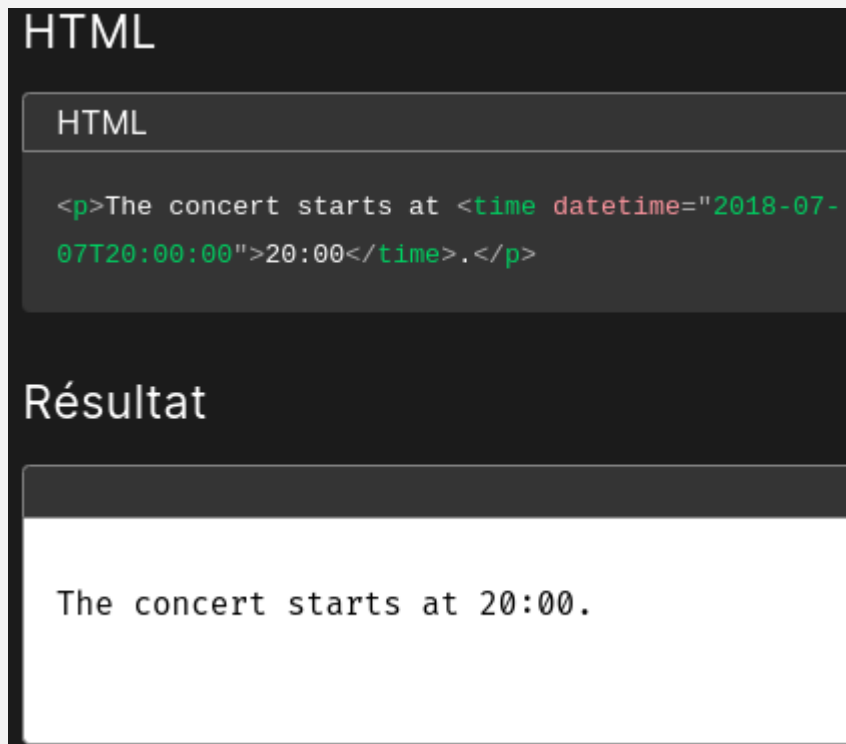
i. L'inspecteur

Cet outil est disponible sur presque tous les navigateurs. Elle permet de trouver les problèmes dans votre page web, de mesurer la performance de votre site web.

6. Quelques balises à savoir:

Voici quelques balises essentiels à savoir:

- address: Permet de renseigner les informations concernant une personne ou une organisation.
- time: Permet de présenter des dates et heures dans un format lisible par machine.



- abbr: L'élément représente une abréviation ou un acronyme.

- ol: Permet de représenter une liste ordonnée d'éléments.

```
1 <ol>
2   <li>Mix flour, baking powder, sugar, and salt.</li>
3   <li>In another bowl, mix eggs, milk, and oil.</li>
4   <li>Stir both mixtures together.</li>
5   <li>Fill muffin tray 3/4 full.</li>
6   <li>Bake for 20 minutes.</li>
7 </ol>
8
```

OUTPUT

1. Mix flour, baking powder, sugar, and salt.
2. In another bowl, mix eggs, milk, and oil.
3. Stir both mixtures together.
4. Fill muffin tray 3/4 full.
5. Bake for 20 minutes.

- dl: Représente une liste de groupe de termes et de descriptions.
- cite: Utilisé pour baliser le titre d'une création citée.
- em: Marque le texte qui met l'accent sur le stress.

CSS

1. Présentation

CSS(Cascading Style Sheets) c'est un langage qui permet d'apporter du style au page web.

CSS décrit comment les éléments HTML seront affichés sur l'écran, papier ou d'autre média. De plus, CSS peut contrôler la mise en page de plusieurs pages web.

2. Structure d'un fichier css

a. Structure

Pour avoir un fichier css il faut créer un fichier avec l'extension `.css` et ensuite le mettre dans le même dossier que le fichier `.html` pour démarrer.

Dans un fichier css nous pouvons apercevoir que les codes css s'écrivent entre des accolades `{ }`, les *sélecteurs* apparaissent en vert, les *propriétés* en rouge et les *valeurs* en bleu.

```
body {  
  background-color: lightblue;  
}  
  
h1 {  
  color: white;  
  text-align: center;  
}
```

b. C'est quoi un sélecteur?

Un sélecteur permet de cibler un élément HTML afin de lui appliquer nos mise en forme spécifiées dans notre code CSS.

```
h1 {  
  color: blue;  
  background-color: yellow;  
}  
  
p {  
  color: red;  
}
```

Quand on veut appliquer le même style sur plusieurs éléments distincts, il est possible de combiner les sélecteurs individuels en une liste.

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

i. Types de sélecteurs:

Sélecteurs de type, de classe, et d'ID:

On retrouve dans ce groupe les sélecteurs ciblant les éléments HTML:

```
h1 {  
  
}
```

On trouve aussi les sélecteurs ciblant une classe:

```
.window{  
  
}
```

Et enfin l'ID:

```
#unique{  
  
}
```

Sélecteurs d'attribut:

Ce groupe de sélecteur permet d'attribuer un style sur un élément en fonction de la présence d'un attribut donnée (avec ou sans une valeur précise) sur l'élément cible:

```
a[title]{  
  
}  
  
a[href="https://topinfo.com"] {  
  
}
```

Ici on est en présence d'un élément HTML <a> qui dispose d'un attribut title.

Sélecteurs pseudo-classes:

Une pseudo-classe peut être ajoutée à un sélecteur afin de préciser l'état dans laquelle doit être l'élément HTML afin de lui affecter une mise en forme.

```
a:hover {  
    background-color: yellow;  
}
```

La pseudo-classe :hover permet d'appliquer une mise en forme lorsque l'utilisateur survole l'élément HTML <a>.

La liste des pseudo-classes est longue je vous invite à visiter le lien <https://developer.mozilla.org/fr/docs/Web/CSS/Pseudo-classes>

Sélecteurs pseudo-éléments:

Un pseudo-élément permet de mettre en forme certaines parties de l'élément ciblé.

```
p::first-line {  
    background-color: yellow;  
}
```

Le pseudo-élément ::first-line permettra de ne cibler que la première ligne de chaque élément paragraphe <p>.

Sélecteurs enfant:

Le symbole ">" sépare deux sélecteurs et cible seulement les éléments du second sélecteur qui sont des enfants directes du premier sélecteur.

```
div > p {  
    background-color: yellow;  
}
```

Pour tous les éléments <p> enfants de l'élément <div> on appliquera une couleur de fond jaune.

3. Mettre du css dans une page HTML

Il y a plusieurs façon d'écrire du code CSS:

- Dans un fichier css externalisé:

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="mystyle.css">
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Dans l'exemple ci-dessus, il est important de savoir que le fichier mystyle.css se trouve dans le même dossier que le fichier html.

- Dans un attribut style d'un élément HTML:

```
<!DOCTYPE html>
<html>
<body>
  <h1 style="color:blue;text-align:center;">This is a
  heading</h1>
  <p style="color:red;">This is a paragraph.</p>
</body>
</html>
```

Directement dans un fichier html

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      body {
        background-color: linen;
      }
      h1 {
        color: maroon;
        margin-left: 40px;
      }
    </style>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

4. Outil de débogage dans un navigateur

A voir en pratique.

5. Modèle de boîte en css

En CSS, tout élément est inclus dans une boîte. Il existe cependant deux types de boîtes: *les boîtes en bloc* et *les boîtes en ligne*.

a. Les boîtes en bloc:

- La boîte s'étend automatiquement sur toute la largeur de son conteneur.
- La boîte crée un retour à la ligne et met les éléments suivants à la ligne d'après.
- Les propriétés `width` et `height` sont toujours respectées

Les éléments tels que `<h1>`, `<h2>`,... et les paragraphes `<p>` utilisent par contre le mode bloc par défaut.

b. Les boîtes en ligne:

- La boîte ne crée pas de retour à la ligne.
- Les propriétés `width` et `height` ne s'appliquent pas.
- Le `padding`, les marges et les bordures verticales ne déplaceront pas les éléments aux alentours.
- Le `padding`, les marges et les bordures horizontales seront appliquées et déplaceront les éléments aux alentours.

Exemple:

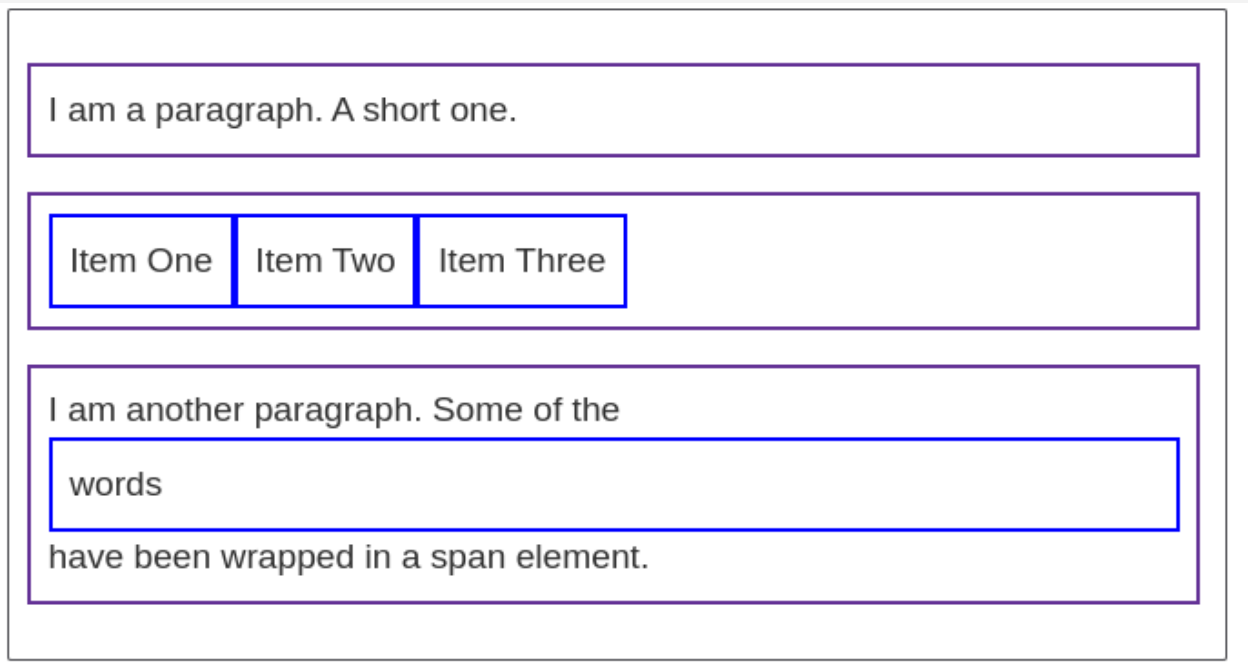
```
p,
ul {
  border: 2px solid rebeccapurple;
  padding: .5em;
}

.block,
li {
  border: 2px solid blue;
  padding: .5em;
}

ul {
  display: flex;
  list-style: none;
}

.block {
  display: block;
}
```

```
<p>I am a paragraph. A short one.</p>
<ul>
  <li>Item One</li>
  <li>Item Two</li>
  <li>Item Three</li>
</ul>
<p>I am another paragraph. Some of the <span
class="block">words</span> have been wrapped in a
<span>span element</span>.</p>
```



Ci-dessus, nous avons ici trois éléments différents qui sont tous en positionnement extérieur “en bloc”. Pour le premier élément le navigateur va le disposer comme une boîte “en bloc”, il occupera alors sa propre nouvelle ligne et s’étend en largeur pour occuper toute l’espace libre.

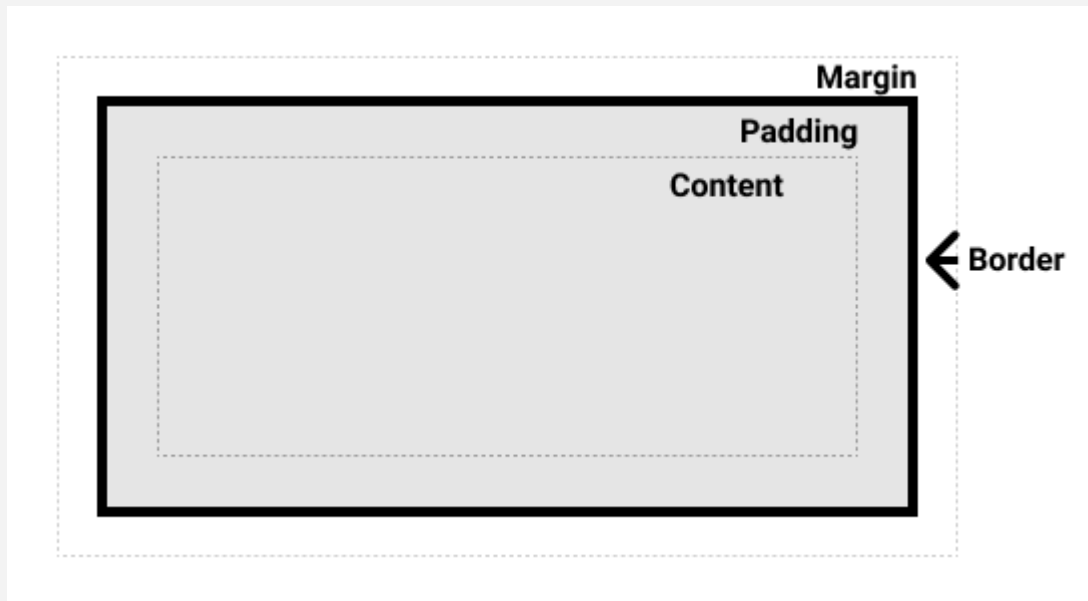
Le deuxième élément est une liste `<ul>`, sur laquelle on a appliqué une règle `display: flex`, pour tous les éléments contenus dans la liste. Même si par défaut une liste est en disposition “en bloc”, elle s’étend en largeur à cause de la règle qu’on lui a appliqué ci-dessus.

Pour le troisième élément, elle est disposée en bloc dans lequel on trouve deux éléments `<span>`. Par défaut, ces deux span sont disposés en ligne mais on a appliqué une règle pour la deuxième à l’aide de la classe nommée “block” qui lui attribue la propriété `display: block`; ce qui explique pourquoi elle occupe toute la ligne.

c. Les composants d'une boîte

- La boîte de contenu: C’est la zone dans laquelle sont affichés nos éléments contenus dans notre boîte et qui peut être dimensionnée par les propriétés CSS `width` et `height`.
- La boîte de padding (marge intérieure): C’est une zone qui représente un espace encadrant le contenu. Sa taille peut être contrôlée en utilisant la propriété `padding`.
- La boîte de bordure: Ce composant englobe le contenu et le padding pour former une bordure. Sa taille et son style peuvent être manipulés à partir du propriété CSS `border`.

- La boîte de marge: C'est le composant qui englobe le contenu, le padding et la bordure. C'est aussi un espace vierge à l'extérieur séparant l'élément des autres éléments de la page. Sa taille et son style peuvent être manipulés à partir du propriété CSS margin.



d. Les modèles de boîtes

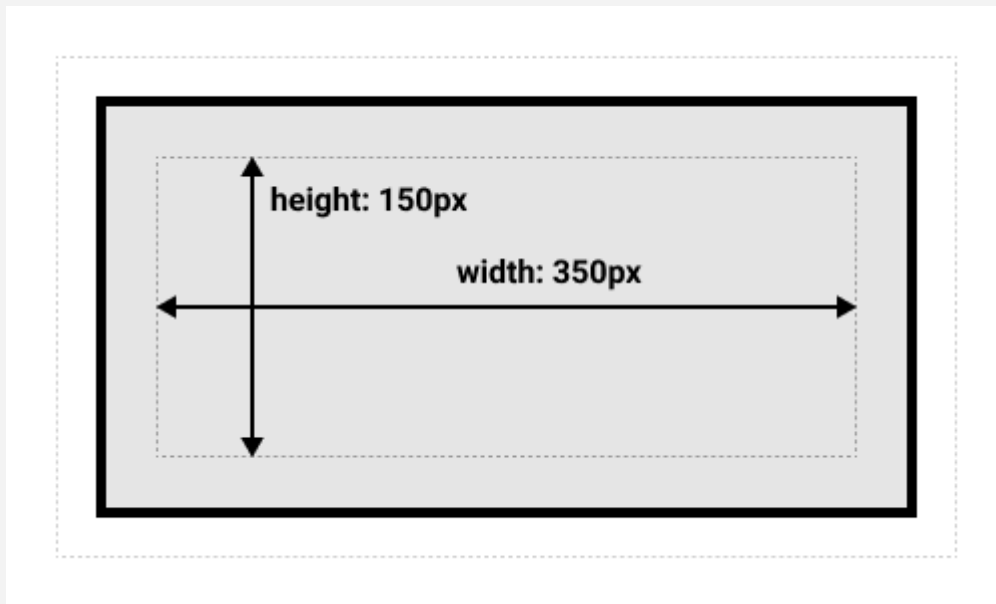
- Modèle de boîte CSS standard:

Dans ce modèle lorsqu'on définit les propriétés de `width` et `height`, celles-ci définissent la largeur et l'hauteur de la boîte de contenu. Par conséquent, la taille du padding et de la bordure s'ajouteront à la largeur et à la hauteur définies par `width` et `height`. Par contre la marge n'est pas prise en compte car elle est extérieure.

```
CSS

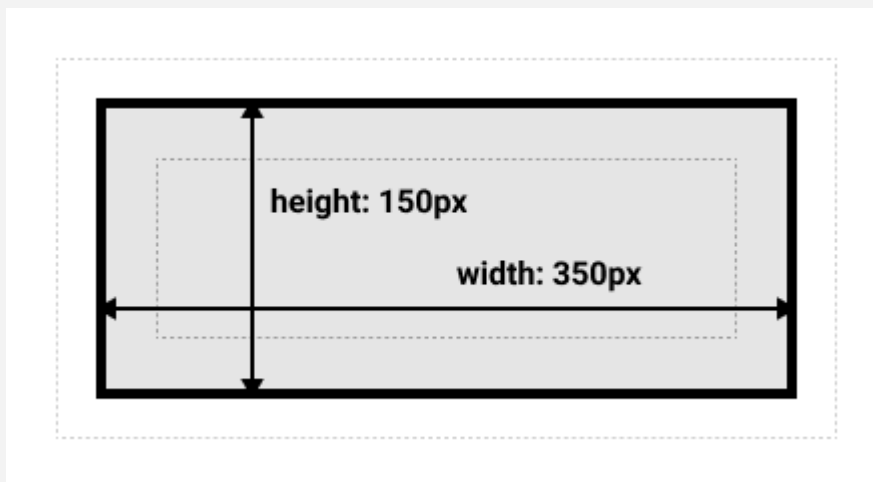
.box {
  width: 350px;
  height: 150px;
  margin: 10px;
  padding: 25px;
  border: 5px solid black;
}
```

Pour notre exemple donc l'espace occupé en largeur est égale à 410 px = 350 px + 25 px + 25 px + 5px + 5px et la hauteur , 210 px = 150 px + 25 px + 25 px + 5 px + 5 px. Il faut ajouter les valeurs de padding et border deux fois car il y en a dans les deux extrémités.



- Modèle de boîte CSS alternatif.

Ce modèle permet de ne pas avoir à calculer les valeurs des propriétés `width` et `height` car elles définissent déjà la valeur totale de la boîte en comprenant le contenu, le padding et la bordure.



Par défaut le navigateur utilise le modèle standard, donc pour utiliser le modèle alternatif, il faut définir la propriété `box-sizing: border-box;`

Pour utiliser le modèle alternatif dans l'intégralité de vos pages il faut spécifier au plus haut niveau ce propriété:

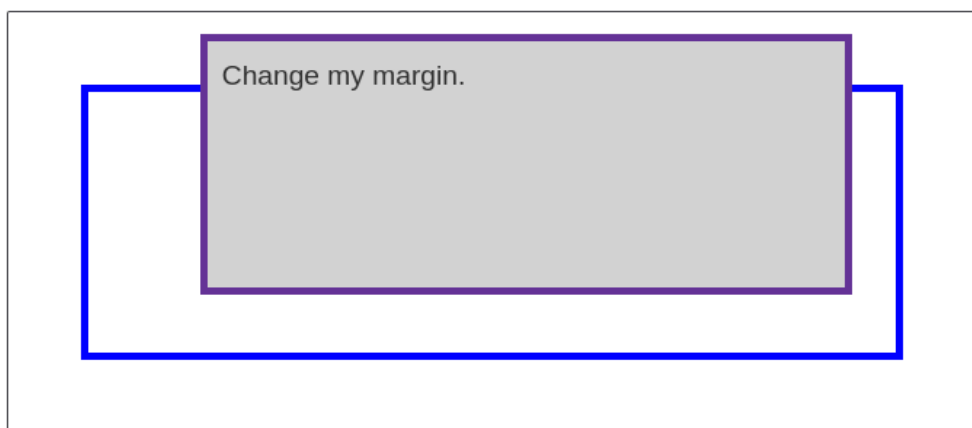
```
html {
  box-sizing: border-box;
}
*, *::before, *::after {
  box-sizing: inherit;
}
```

e. Les marges

Les marges sont des zones d'espacement invisible qui encadrent votre boîte et repoussent les éléments alentour de celle-ci. Il est possible de fixer les quatres valeurs d'une marge d'un seul coup à l'aide de la propriété "**margin**". Voici les quatres propriétés en question:

- margin-top;
- margin-right;
- margin-bottom;
- margin-left;

Exemple:



Interactive editor

```
.box {
  margin-top: -40px;
  margin-right: 30px;
  margin-bottom: 40px;
  margin-left: 4em;
}
```

```
<div class="container">
  <div class="box">Change my margin.</div>
</div>
```

f. Les bordures

Pour agir sur les bordures il existe plusieurs propriétés qui permettent de régler le style, la taille et la couleur pour chacun des quatres côtés.

Propriété pour agir sur les quatres côtés:

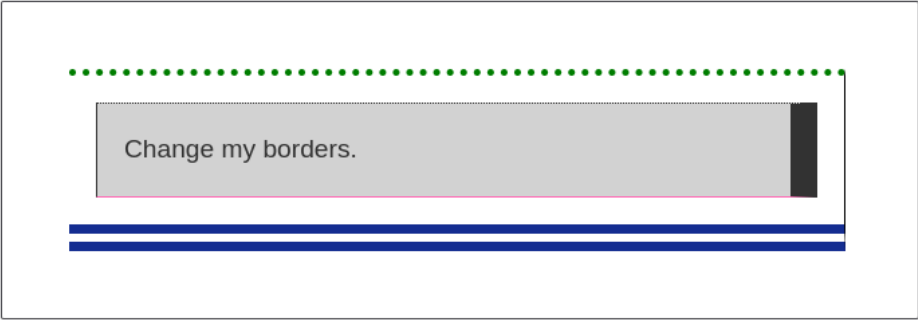
- border-top;
- border-right;
- border-bottom;
- border-left;

Pour modifier la taille, le style et la couleur de tous les côtés:

- border-width;
- border-style;
- border-color;

Il est aussi possible de modifier la taille, le style ou la couleur d'un seul côté:

- border-top-width;
- border-right-width;
- border-bottom-width;
- border-left-width;
- ...



Interactive editor

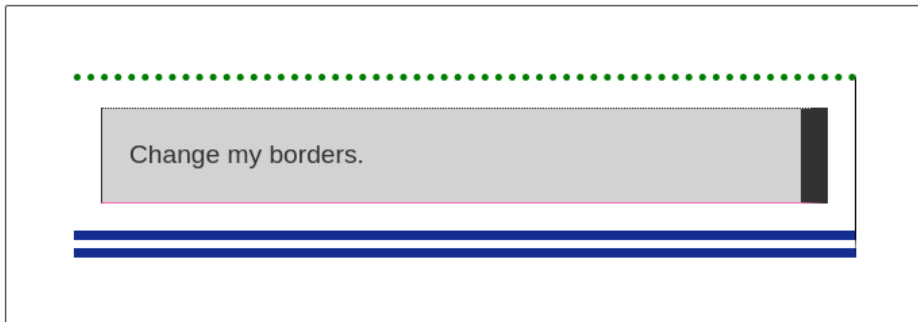
```
.container {  
  border-top: 5px dotted green;  
  border-right: 1px solid black;  
  border-bottom: 20px double rgb(23,45,145);  
}  
  
.box {  
  border: 1px solid #333333;  
  border-top-style: dotted;  
  border-right-width: 20px;  
  border-bottom-color: hotpink;  
}
```

```
<div class="container">  
  <div class="box">Change my borders.</div>  
</div>
```


g. Le padding

Le padding se situe entre la bordure et le contenu. On ne peut pas attribuer une valeur numérique négative avec un padding. Voici les quatres propriétés:

- padding-top;
- padding-right;
- padding-bottom;
- padding-left;



Interactive editor

```
.container {  
  border-top: 5px dotted green;  
  border-right: 1px solid black;  
  border-bottom: 20px double rgb(23,45,145);  
}  
  
.box {  
  border: 1px solid #333333;  
  border-top-style: dotted;  
  border-right-width: 20px;  
  border-bottom-color: hotpink;  
}
```

```
<div class="container">  
  <div class="box">Change my borders.</div>  
</div>
```

6. Les polices d'écritures

a. Comment utiliser une police d'écriture classique?

Les polices d'écriture en HTML sont appliquées par la propriété `font-family`.

CSS

```
p {  
  font-family: Helvetica, "Trebuchet MS", Verdana, sans-serif;  
}
```

La pile de police permet de préciser la police préférable, puis l'alternative sûre pour le web et enfin celle par défaut du système.

b. Importer une police d'écriture en locale

La première étape est de rechercher les polices de caractère. *Il faut être extrêmement prudent dans l'utilisation des polices de caractère car elles sont soumises à des droits d'utilisation.* Heureusement, il y a des polices de caractère gratuites <https://www.fontsquirrel.com/>, <http://www.dafont.com/>, <https://everythingfonts.com/>, <https://fonts.google.com/>

Pour cette exemple nous allons chercher des polices de caractères sur le site <https://www.fontsquirrel.com/>.

Après avoir trouvé la police de votre choix, appuyer sur le bouton de téléchargement et enregistrez le fichier dans le même répertoire que les fichiers html et css.

Pour une question d'organisation, il faudrait mettre les fichiers de police dans un répertoire "fonts" comme sous répertoire au même niveau que les fichiers html et css.

Voici le code css:

```
@font-face {
  font-family: 'ciclefina';
  src: url('fonts/cicle_fina-webfont.woff2') format('woff2'),
       url('fonts/cicle_fina-webfont.woff') format('woff');
  font-weight: normal;
  font-style: normal;
}

@font-face {
  font-family: 'zantrokeregular';
  src: url('fonts/zantroke-webfont.woff2') format('woff2'),
       url('fonts/zantroke-webfont.woff') format('woff');
  font-weight: normal;
  font-style: normal;
}
```

Appliquer la police de caractère:

```
h1,h2 {
  font-family: 'zantrokeregular', serif;
  letter-spacing: 0.2rem;
}

p {
  font-size: 1.6rem;
  line-height: 1.6;
  letter-spacing: 0.1rem;
  word-spacing: 0.3rem;
  font-family: 'ciclefina', sans-serif;
}
```

Importer le fichier css dans notre page html

```
<!DOCTYPE html>
<html lang="en-us">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width">
    <title>Web font example</title>
    <link href="web-font-finished.css" rel="stylesheet" type="text/css">
  </head>
  <body>
    <h1>Hipster ipsum is the best</h1>

    <p>Tacos actually microdosing, pour-over semiotics banjo chicharrones

    <p>Cray food truck brunch, XOXO +1 keffiyeh pickled chambray waistcoa

    <h2>It is the quaintest</h2>

    <p>Bespoke green juice aesthetic leggings DIY williamsburg selvage. B

    <h2>No, really...</h2>

    <p>Trust fund celiac farm-to-table PBR&B. Brunch art party mumble

    <p>Taxidermy tofu YOLO, sustainable etsy flexitarian art party stumpt
  </body>
</html>
```

c. Importer une police d'écriture distante

- Allez sur <https://www.google.com/fonts>.
- Utilisez les filtres sur la droite pour afficher les types de polices à choisir et retenez une paire de fontes qui vous plaisent.
- Pour sélectionner une famille de fontes, pressez le bouton ⊕ sur le côté.
- Après avoir choisi les familles de fontes, pressez la barre avec [Nombre] Families Selected en bas de la page.
- Dans l'écran résultant, copiez d'abord la ligne de code HTML affichée et collez-la dans l'en-tête de votre fichier HTML. Mettez-la au-dessus de l'élément <link> existant, de sorte que la police soit importée avant que le navigateur essaye de l'utiliser dans la CSS.
- Copiez ensuite les déclarations CSS listées dans la CSS comme il convient d'appliquer la fonte personnalisée à votre HTML.

7. Les couleurs et unités de mesures

En général, on peut appliquer une couleur à partir de la propriété `color`. Les couleurs en css peuvent être spécifiés de manière différentes:

- Par le nom de la couleur (en anglais):

```
#myDIV {  
    color: blue; /* Blue text color */  
    border: 10px solid currentcolor; /* Blue border color */  
}
```

- Avec des valeurs RGB (Red, Green, Blue):

```
#myDIV {  
    color: rgb(255,0,0); /* red text color */  
    border: 10px solid currentcolor; /* Blue border color */  
}
```

- En hexadécimal:

```
#myDIV {  
    color: #FF0000; /* red text color */  
    border: 10px solid currentcolor; /* Blue border color */  
}
```

8. Positionnement

a. Pixel ou pourcentage

Il est possible d'utiliser comme unité de mesure pixel ou pourcentage.

```
<!Doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Pourcentage</title>
    <link rel="stylesheet" href="pourcentage.css">
  </head>
  <body>
    <div class="taillefixe">Fixe</div>
    <div class="taillevariable">Variable</div>
  </body>
</html>
```

```
.taillefixe{ width: 1000px; border: 2px solid #cb3d3d; }
.taillevariable{ width: 100%; border: 2px solid #336699; }
```

En utilisant le pourcentage comme valeur, l'affichage sur l'écran s'adapte selon la taille de cette dernière.

b. Position

La propriété en CSS `position` positionne un élément dans un document (page web). Les propriétés `top`, `right`, `bottom` et `left` représentent l'emplacement final de l'élément positionné.

Syntaxe:

CSS

```
position: static;
position: relative;
position: absolute;
position: fixed;
position: sticky;

/* Valeurs globales */
position: inherit;
position: initial;
position: revert;
position: unset;
```

❖ static:

C'est le comportement par défaut. L'élément est positionné dans le flux avec sa position.

❖ relative:

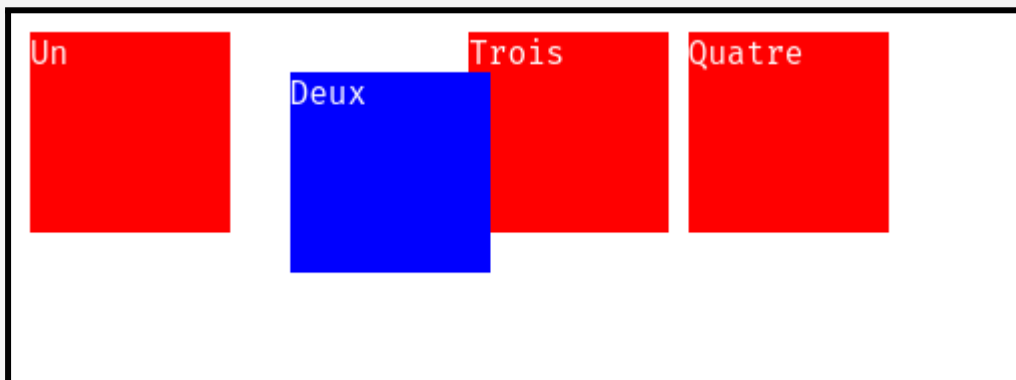
L'élément est positionné dans le flux normal du document puis déplacé selon les valeurs fournies par `top`, `right`, `bottom`, `left`.

HTML

```
<div class="box" id="un">Un</div>
<div class="box" id="deux">Deux</div>
<div class="box" id="trois">Trois</div>
<div class="box" id="quatre">Quatre</div>
```

CSS

```
* {  
  box-sizing: border-box;  
}  
  
.box {  
  display: inline-block;  
  width: 100px;  
  height: 100px;  
  background: red;  
  color: white;  
}  
  
#deux {  
  position: relative;  
  top: 20px;  
  left: 20px;  
  background: blue;  
}
```



❖ absolute

Les éléments positionnés de façon absolue sont retirés du flux normal du document et ne prennent plus d'espace et se positionnent par rapport à son plus proche ancêtre positionné.

HTML

Play ↗



```
<h1>Positionnement absolu</h1>
```

```
<p>Un élément de bloc simple. Les éléments de bloc adjacents sont sur de nouvelles  
lignes en dessous.</p>
```

```
<p class="positioned">Par défaut, on occupe 100% de la largeur de l'élément parent et on  
est aussi grand que notre contenu.</p>
```

```
<p>Nous sommes séparés par nos marges (une seule marge en raison de la fusion des  
marges).</p>
```

```
<p>Les éléments <em>en ligne (inline)</em> <span>comme celui-ci</span> et <span>celui-  
là</span> se situent sur la même ligne avec également les nœuds texte. S'il y a de  
l'espace sur la même ligne. Les éléments qui dépassent <span>passent à la ligne si  
possible – comme pour ce texte.</span> ou cette image </p>
```

CSS

```
* {
  box-sizing: border-box;
}

body {
  width: 500px;
  margin: 0 auto;
}

p {
  background: aqua;
  border: 3px solid blue;
  padding: 10px;
  margin: 10px;
}

span {
  background: red;
  border: 1px solid black;
}

.positioned {
  position: absolute;
  background: yellow;
  top: 30px;
  left: 30px;
}
```

Positionnement absolu

Par défaut, on occupe 100% de la largeur de l'élément parent et on est aussi grand que notre contenu.

Un élément de bloc simple. Les éléments de bloc adjacents sont sur de nouvelles lignes en dessous.

Nous sommes séparés par nos marges (une seule marge en raison de la fusion des marges).

Les éléments *en ligne (inline)* comme celui-ci et celui-là se situent sur la même ligne avec également les nœuds texte. S'il y a de l'espace sur la même ligne. Les éléments qui dépassent passent à la ligne si possible – comme pour ce texte. ou cette image



❖ fixed:

L'élément est retiré du flux normal et aucun espace n'est laissé pour l'élément.

❖ sticky

Ce positionnement est un mélange entre le relatif et le fixe. L'élément est positionné en relatif jusqu'à ce qu'un seuil soit franchi. Quand le seuil est franchi, l'élément est positionné en fixe.

9. Responsive design

Le concept de **Responsive Web Design** permet aux pages web de modifier leur disposition et leur apparence pour s'adapter à différentes largeurs d'écran.

a. Média queries

Les **Media Queries** nous permettent d'effectuer une série de tests (point de rupture/break points) et d'appliquer le css de manière sélective pour donner à la page le style approprié.

CSS

```
@media screen and (min-width: 800px) {  
  .container {  
    margin: 1em 2em;  
  }  
}
```

L'exemple ci-dessus nous montre le type de media queries qui peut être utilisé pour tester si la page Web actuelle est affichée comme média d'écran (donc pas un document à imprimer) et si la fenêtre de visualisation a au moins 800 pixels de large. Si cette condition est vérifiée le css du sélecteur `.container` sera appliqué.

```
@media screen and (max-width: 600px) {  
  div.example {  
    display: none;  
  }  
}
```

Dans l'exemple ci-dessus par contre testera si la page Web est affichée comme média à l'écran et si la fenêtre de visualisation a au maximum 600 pixels. Si la condition est vérifiée l'élément `<div>` avec le sélecteur `.example` sera caché.

```
@media only screen and (orientation: landscape) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Dans l'exemple ci-dessus on change la couleur de fond de la page web si l'orientation du navigateur est en landscape.

```
/* When the width is between 600px and 900px OR above 1100px - change the appearance of <div> */  
@media screen and (max-width: 900px) and (min-width: 600px), (min-width: 1100px) {  
  div.example {  
    font-size: 50px;  
    padding: 50px;  
    border: 8px solid black;  
    background: yellow;  
  }  
}
```

Ici on combine différentes conditions avant d'appliquer un style. Si la largeur de la fenêtre de visualisation est comprise entre 600 pixels *et* 900 pixels *ou* supérieure à 1100 pixels on change l'apparence de l'élément `div`.

b. Images responsives

CSS

```
img {  
  max-width: 100%;  
}
```

Ceci est l'approche la plus simple qui permet d'adapter l'affichage de notre image quelque soit la taille de l'écran sur laquelle notre visiteur visite notre page Web.

Cependant, elle présente des inconvénients évidents:

- gaspillage de la bande passante car l'image est affichée à une taille beaucoup plus petite que sa taille réelle.
- Un utilisateur mobile peut télécharger une image dont la taille est beaucoup plus grande que ce qu'il voit réellement dans la fenêtre.

i. Image responsive avec un width à 100%

La méthode la plus simple est de mettre la propriété `width` à 100%, l'image prendra toujours 100% de la largeur de son parent.

Il est toujours possible d'utiliser les Media Queries pour établir des break points et de contrôler l'affichage de l'image.

ii. Commutation de résolution: tailles différentes

Nous voulons afficher la même image mais dont la taille s'adapte selon l'appareil (l'écran sur lequel notre visiteur regarde notre site). Avec la balise `<img>` on ne peut spécifier qu'une seule source d'image avec l'attribut `src`. Par contre, avec l'attribut `srcset` et `sizes` il est possible de fournir plusieurs images sources avec des indications qui permettent au navigateur de faire le bon choix.

HTML

```

```

`srcset="nom_image largeur_intrinsèque_pixels(notez en w)"`

`largeur_intrinsèque_pixels`: c'est la taille réelle de l'image.

`sizes`: Définit un ensemble de conditions pour le média.

iii. Commutation de résolution: même taille, résolutions différentes

Permet au navigateur de choisir une image de résolution appropriée en utilisant `srcset` avec `x-descriptors` et sans `sizes`.

HTML

```

```

iv. Le méta-tag du viewport

Pour que les navigateurs mobiles sachent qu'ils doivent ajuster la largeur de la fenêtre à la largeur d'écran de l'appareil et mettre l'échelle du document à 100% de sa taille prévue il faut ajouter la syntaxe suivante entre la balise `<head>` de votre page HTML:

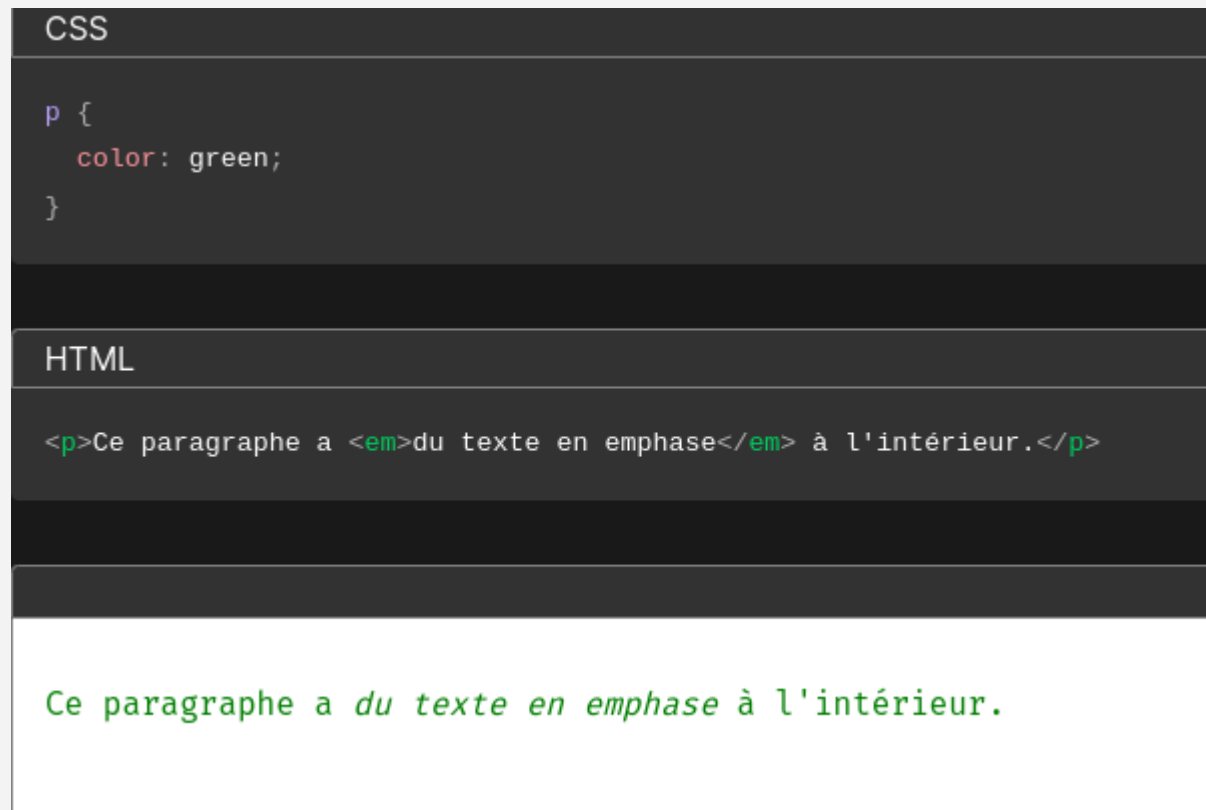
HTML

```
<meta name="viewport" content="width=device-width,initial-scale=1">
```

10. L'héritage

a. Qu'est ce que l'héritage en css?

L'héritage contrôle ce qui se produit lorsqu'aucune valeur n'est indiquée pour une propriété d'un élément. C'est ce qui se produit dans l'image suivante:



Il est aussi possible que les éléments héritent d'un style prédéfini particulier. Par exemple:

```
<!Doctype html>  
<html>  
  <head>  
    <meta charset="utf-8">  
    <title>Ma Page</title>  
    <link rel="stylesheet" href="heritage.css">  
  </head>  
  <body>  
  
    <h1> h1 </h1>  
  
    <p> p </p>  
  
    <span> span </span>  
  
    <div> div </div>  
  
  </body>  
</html>
```

Ce qui donnera dans le navigateur:



On remarque que la balise h1 hérite d'une marge et d'une certaine taille de caractères alors que nous n'avons rien demandé. C'est pour cela qu'on utilise parfois un RESET CSS de manière à remettre à zéro les styles.

b. Comment fonctionne le reset css?

Remettre les styles à zéro pour certaines balises qui ont par défaut des styles. Cette méthode provoque parfois des comportements inattendus, par conséquent il faut être extrêmement vigilant pour l'utiliser.

```
section {
  display: block;
}
body {
  line-height: 1;
}
ol,
ul {
  list-style: none;
}
```

La solution alternative serait d'utiliser une feuille de style de base, cette dernière ne devra pas annuler les styles par défaut des différents navigateurs mais les harmoniser

11. Présentation de bootstrap

Bootstrap c'est un ensemble qui contient des codes HTML et CSS qui permettent de faciliter la création du design de sites et d'applications web. C'est un framework de frontend écrit avec les langages de programmation: CSS, Javascript, Sass, Less.

12. Exemple d'utilisation bootstrap

Voici un exemple qui permet d'appliquer les styles définis dans bootstrap dans une page web. Ici nous utilisons la ressource en ligne de bootstrap (CDN: Content Delivery Network).

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-9ndCyUaIbzAi2FUVXJi0CjmCapSm07SnpJef0486qhLnuZ2cdeRh002iuK6FUUVM"
crossorigin="anonymous">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
integrity="sha384-geWF76RCwLtnZ8qwWowPQNguL3RmwHVBC9FhGdlKrxdiJJigb/j/68SIy3Te4Bkz"
crossorigin="anonymous"></script>
  </body>
</html>
```

On peut accéder à plusieurs exemples sur le site officielles de bootstrap <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Il est aussi possible d'utiliser bootstrap en local sans avoir de connexion internet.

JAVASCRIPT

1. Présentation de JAVASCRIPT

JavaScript est une langage de script, multi-platform et orienté objet. JavaScript est un langage faiblement typé.

Pour commencer à développer en JS il suffit d'avoir un navigateur web récent.

2. Types et grammaire

Les bases du langage

JS est *sensible à la casse*. On peut donc utiliser comme nom de variable `früh`:

JS

```
var früh = "toto";  
typeof Früh; // undefined car JavaScript est sensible à la casse
```

En JS les instructions sont séparées par des points-virgules.

Commentaires

Pour ajouter des commentaires on utilise `//`, `/* */`

JS

```
// un commentaire sur une ligne  
  
/* un commentaire plus  
   long sur plusieurs lignes  
*/  
  
/* Par contre on ne peut pas /* imbriquer des commentaires */ SyntaxError */
```

Déclarations

Il existe trois types de déclarations de variable en JS:

var: On déclare une variable en initialisant sa valeur.

let: On déclare une variable dont la portée est celle du bloc courant.

const: On déclare une constante nommée.

JS

```
var a;  
console.log("La valeur de a est " + a); // La valeur de a est undefined  
  
console.log("La valeur de b est " + b); // La valeur de b est undefined  
var b; // La déclaration de la variable est "remontée" (voir la section ci-après)  
  
console.log("La valeur de x est " + x); // signale une exception ReferenceError  
let x;  
let y;  
console.log("La valeur de y est " + y); // La valeur de y est undefined
```

Structure de données et types

Les tableaux:

Un tableau est une liste de zéro ou plusieurs expressions qui représentent les éléments du tableau, encadrées par des crochets `[]`.

JS

```
var cafés = ["Brésilien", "Colombien", "Kona"];
```

Les tableaux sont également appelés des objets `Array`.

Les booléens:

Le type booléen possède deux valeurs littérales: `true` et `false`.

Les nombres décimaux:

Les nombres décimaux peut être composés de ces différentes parties:

- Un entier, pouvant être signé (précédé d'un « + » ou d'un « - »),
- Un point, séparateur décimal (« . »),
- La partie décimale (un autre nombre)
- Un exposant.

```
3.14
2345.789
.333333333333333333
```

Les objets:

Un objet c'est une liste de zéro ou plusieurs propriétés définies par des paires de noms/valeurs délimitées par des accolades { }.

JS

```
var soldes = "Toyota";

function carTypes(nom) {
  return (nom === "Honda") ?
    nom :
    "Désolé, nous ne vendons pas de " + nom + "." ;
}

var voiture = { maVoiture: "Saturn", getVoiture: carTypes("Honda"), spécial: soldes };

console.log(voiture.maVoiture); // Saturn
console.log(voiture.getVoiture); // Honda
console.log(voiture.spécial); // Toyota
```

Le premier élément de l'objet voiture définit une propriété maVoiture avec comme valeur "Saturn".

JS

```
var voiture = { plusieursVoitures: {a: "Saab", b: "Jeep"}, 7: "Mazda" };

console.log(voiture.plusieursVoitures.b); // Jeep
console.log(voiture[7]); // Mazda
```

3. Instructions conditionnelles, boucles et itérations

Instruction if...else

JS

```
if (condition_1) {  
    instruction_1;  
} else if (condition_2) {  
    instruction_2;  
} else if (condition_n) {  
    instruction_n;  
} else {  
    dernière_instruction;  
}
```

Switch

JS

```
switch (fruit) {  
    case "Orange":  
        console.log("Les oranges sont à 60 centimes le kilo.");  
        break;  
    case "Pomme":  
        console.log("Les pommes sont à 32 centimes le kilo.");  
        break;  
    case "Banane":  
        console.log("Les bananes sont à 48 centimes le kilo.");  
        break;  
    case "Cerise":  
        console.log("Les cerises sont à 3€ le kilo.");  
        break;  
    case "Mangue":  
        console.log("Les mangues sont à 50 centimes le kilo.");  
        break;  
    default:  
        console.log("Désolé, nous n'avons pas de " + fruittype + ".");  
}  
console.log("Souhaitez-vous autre chose ?");
```

Gestion des erreurs et des exceptions

L'instruction `try...catch` nous permet de définir un bloc d'instructions qu'on exécutera en cas d'erreur lorsqu'une exception se produit

JS

```
try {
  throw "monException"; // on génère une exception
} catch (e) {
  // les instructions utilisées pour gérer les exceptions
  enregistrerErreurs(e); // on passe l'objet représentant l'exception à une fonction
  utilisée pour gérer les erreurs
}
```

Boucles et itérations:

L'instruction `for`:

```
let str = '';
for (let i = 0; i < 9; i++) {
  str = str + i;
}

console.log(str);
// Expected output: "012345678"
```

L'instruction `do...while`:

JavaScript Demo: Statement - Do...While

```
1 let result = '';
2 let i = 0;
3
4 do {
5   i = i + 1;
6   result = result + i;
7 } while (i < 5);
8
9 console.log(result);
10 // Expected output: "12345"
11
```

L'instruction while:

```
JavaScript Demo: Statement - While
1 let n = 0;
2
3 while (n < 3) {
4   n++;
5 }
6
7 console.log(n);
8 // Expected output: 3
9
```

L'instruction break:

```
JavaScript Demo: Statement - Break
1 let i = 0;
2
3 while (i < 6) {
4   if (i === 3) {
5     break;
6   }
7   i = i + 1;
8 }
9
10 console.log(i);
11 // Expected output: 3
12
```

L'instruction continue:

```
JavaScript Demo: Statement - Continue
1 let text = '';
2
3 for (let i = 0; i < 10; i++) {
4   if (i === 3) {
5     continue;
6   }
7   text = text + i;
8 }
9
10 console.log(text);
11 // Expected output: "012456789"
12
```

L'instruction for...in:

```
JavaScript Demo: Statement - For...In
1  const object = { a: 1, b: 2, c: 3 };
2
3  for (const property in object) {
4    console.log(`${property}: ${object[property]}`);
5  }
6
7  // Expected output:
8  // "a: 1"
9  // "b: 2"
10 // "c: 3"
11
```

L'instruction for...of:

```
JavaScript Demo: Statement - For...Of
1  const array1 = ['a', 'b', 'c'];
2
3  for (const element of array1) {
4    console.log(element);
5  }
6
7  // Expected output: "a"
8  // Expected output: "b"
9  // Expected output: "c"
10
```

4. Les fonctions

Une fonction est construite avec le mot-clé function.

```
JS
function carré(nombre) {
  return nombre * nombre;
}
```

L'appel d'une fonction se fait de la manière suivante:

```
JS
var carré = function (nombre) { return nombre * nombre };
var x = carré(4); //x reçoit la valeur 16
```


5. Les classes

On peut créer une classe à partir du mot-clé `class`

JS

```
// Declaration
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}

// Expression; the class is anonymous but assigned to a variable
const Rectangle = class {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};

// Expression; the class has its own name
const Rectangle = class Rectangle2 {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
};
```

Créer un objet à partir d'une classe:

```
const square = new Rectangle(10, 10);
```

6. Gestion d'évènement en JAVASCRIPT

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<p>Click the button to display the date.</p>

<button onclick="displayDate()">The time is?</button>

<script>
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

<p id="demo"></p>

</body>
</html>
```

La fonction `displayDate` sera lancée si on clique sur le bouton "The time is". Javascript permet d'exécuter du code quand un évènement est détecté.

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The
time is?</button>
```

Ce code permettra de changer le contenu de l'élément avec l'id "demo".

Dans le bout code suivant nous allons ajouter un *event listener* qui écoutera le click qu'un utilisateur fera sur un bouton:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript addEventListener()</h2>

<p>This example uses the addEventListener() method to attach
a click event to a button.</p>

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click",
displayDate);

function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

7. JS Promise

a) Callbacks

Un callback est une fonction passée en paramètre à une autre fonction.

Il est parfois nécessaire d'avoir le contrôle sur l'exécution d'une fonction, c'est là qu'entre en jeu les *callbacks*.

Supposons que nous devons calculer l'addition de deux nombres et d'afficher le résultat.

Voici l'exemple 1:

```
function myDisplayer(some) {  
    document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2) {  
    let sum = num1 + num2;  
    return sum;  
}  
  
let result = myCalculator(5, 5);  
myDisplayer(result);
```

On appelle la fonction `myCalculator` puis on sauvegarde le résultat et on appelle une autre fonction `myDisplayer` pour afficher le résultat.

Dans l'exemple suivant on appelle la fonction `myCalculator` qui appellera la fonction d'affichage `myDisplayer`

```
function myDisplayer(some) {  
    document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2) {  
    let sum = num1 + num2;  
    myDisplayer(sum);  
}  
  
myCalculator(5, 5);
```

Le souci avec la première exemple c'est qu'on est obligé d'appeler deux fonctions pour afficher le résultat.

Avec le deuxième exemple, on ne peut pas empêcher la fonction `myCalculator` d'afficher le résultat.

Nous allons faire appel aux `callbacks` pour améliorer ces deux exemples dans le bout de code suivant:

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>Callback Functions</h2>

<p>The result of the calculation is:</p>
<p id="demo"></p>

<script>
function myDisplayer(something) {
    document.getElementById("demo").innerHTML = something;
}

function myCalculator(num1, num2, myCallback) {
    let sum = num1 + num2;
    myCallback(sum);
}

myCalculator(5, 5, myDisplayer);
</script>

</body>
</html>
```

`myDisplayer` est appelé "callback function" elle est passée en paramètre à la fonction `myCalculator` comme argument.

Voici un autre exemple pour illustrer l'utilisation d'un callback

```
// Create an Array
const myNumbers = [4, 1, -20, -7, 5, 9, -6];

// Call removeNeg with a callback
const posNumbers = removeNeg(myNumbers, (x) => x >= 0);

// Display Result
document.getElementById("demo").innerHTML = posNumbers;

// Keep only positive numbers
function removeNeg(numbers, callback) {
  const myArray = [];
  for (const x of numbers) {
    if (callback(x)) {
      myArray.push(x);
    }
  }
  return myArray;
}
```

b) Promise object

"Je promets un résultat!!!!!"

"Producing code" c'est le code qui mets du temps à s'exécuter

"Consuming code" c'est le code qui doit attendre le résultat renvoyé par le "Producing code"

Un objet "Promise" permet de lier le "Producing code" et "Consuming code"

```
let myPromise = new Promise(function(myResolve, myReject) {  
  // "Producing Code" (May take some time)  
  
  myResolve(); // when successful  
  myReject();  // when error  
});  
  
// "Consuming Code" (Must wait for a fulfilled Promise)  
myPromise.then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

Voici un exemple assez simple:

```
let myPromise = new Promise(function(myResolve, myReject) {  
  setTimeout(function() { myResolve("I love You !!"); }, 3000);  
});  
  
myPromise.then(function(value) {  
  document.getElementById("demo").innerHTML = value;  
});
```

Ce bout de code mettra à jour le contenu de l'élément HTML avec l'id demo après 3 secondes.

L'exemple suivant fera un appel HTTP qui va récupérer le contenu de la page mycar.html.

```
let myPromise = new Promise(function(myResolve, myReject) {
  let req = new XMLHttpRequest();
  req.open('GET', "mycar.html");
  req.onload = function() {
    if (req.status == 200) {
      myResolve(req.response);
    } else {
      myReject("File not Found");
    }
  };
  req.send();
});

myPromise.then(
  function(value) {myDisplayer(value);},
  function(error) {myDisplayer(error);}
);
```


c) JS async/await

async et await permettent de faciliter l'utilisation des promises. Les deux bouts de code suivantes sont identiques:

```
async function myFunction() {  
  return "Hello";  
}
```

=

```
function myFunction() {  
  return Promise.resolve("Hello");  
}
```

Voici comment on doit utiliser promises

```
myFunction().then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

Exemple:

```
async function myFunction() {  
  return "Hello";  
}  
myFunction().then(  
  function(value) {myDisplayer(value);},  
  function(error) {myDisplayer(error);}  
);
```

Il est possible de ne pas retourner d'erreur:

```
async function myFunction() {  
  return "Hello";  
}  
myFunction().then(  
  function(value) {myDisplayer(value);}  
);
```

Syntaxe avec *await*.

On utilise uniquement *"await"* à l'intérieur d'une fonction *async*, ce qui permettra de mettre en pause l'exécution et d'attendre la réponse du *promise*.

```
async function myDisplay() {  
  let myPromise = new Promise(function(resolve, reject) {  
    resolve("I love You !!");  
  });  
  document.getElementById("demo").innerHTML = await myPromise;  
}  
  
myDisplay();
```

Pour simuler un traitement qui prendra du temps en utilisant la syntaxe *setTimeout*

```
async function myDisplay() {  
  let myPromise = new Promise(function(resolve) {  
    setTimeout(function() {resolve("I love You !!");}, 3000);  
  });  
  document.getElementById("demo").innerHTML = await myPromise;  
}  
  
myDisplay();
```

8. JS Modules

JS Modules nous permettra d'avoir du code séparé dans plusieurs fichiers. Par conséquent, la maintenance sera facile.

Voici la syntaxe:

```
<script type="module">  
import message from "./message.js";  
</script>
```

Export:

Il est possible d'avoir des fonctions ou variables dans des fichiers externes en utilisant ce qu'on appelle *Named exports* et *Default Exports*.

- Named exports:

Il y a deux façons de créer un named exports: en ligne ou à la fin du fichier.

En ligne:

```
export const name = "Jesse";  
export const age = 40;
```

A la fin:

```
const name = "Jesse";  
const age = 40;  
  
export {name, age};
```

- Default exports:

On ne peut avoir qu'une seule default exports dans un fichier:

```
const message = () => {  
  const name = "Jesse";  
  const age = 40;  
  return name + ' is ' + age + 'years old.';  
};  
  
export default message;
```

Import:

L'importation d'un module dépend si c'est un named exports ou default exports.

Module named exports:

```
import { name, age } from "./person.js";
```

Module default exports:

```
import message from "./message.js";
```

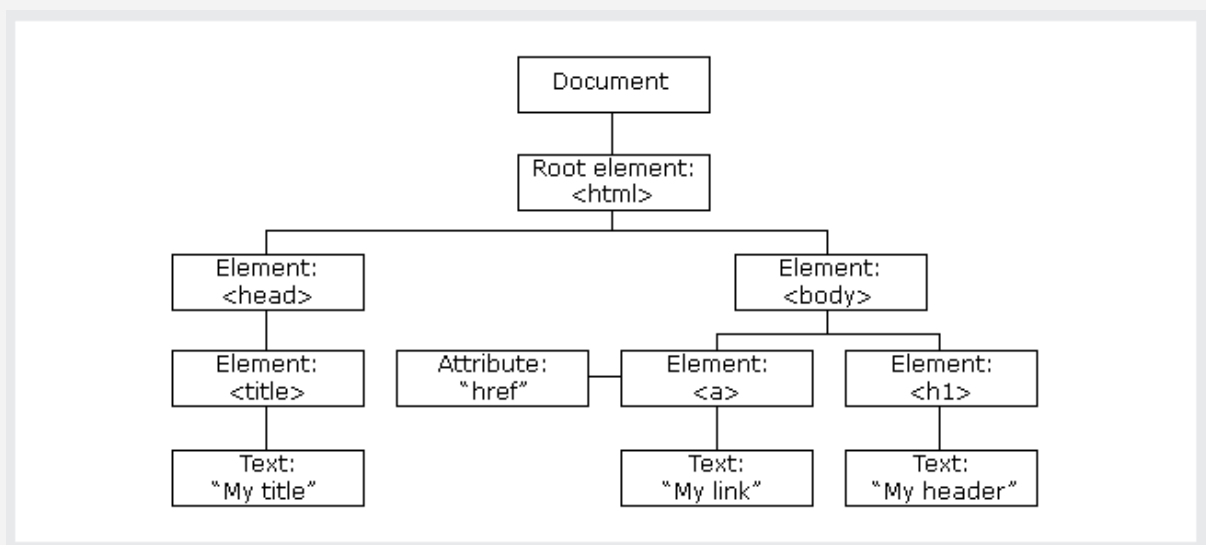
9. JS HTML DOM

a. DOM Introduction

JavaScript peut accéder et changer tous les éléments d'un document HTML.

DOM: Document Object Model

Quand une page web est chargée, le navigateur crée le DOM de la page.



- JS peut changer tous les éléments HTML dans une page;
- JS peut changer tous les attributs HTML dans une page;
- JS peut changer tous les CSS dans une page;
- JS peut supprimer tous les éléments HTML et attributs existants dans une page;
- JS peut ajouter un nouveau élément HTML et attribut;
- JS peut réagir à toutes évènements dans une page;
- JS peut créer un évènement HTML dans une page;

b. DOM Methods

Les DOM methods sont des méthodes ou actions que l'on peut appliquer sur les éléments HTML.

Par contre, une **propriété** est une valeur qu'on peut modifier ou récupérer.

Voici un exemple avec la méthode `getElementById` et la propriété `innerHTML`

```
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

c. DOM Document

L'objet **Document** représente notre page web. Si on veut accéder à un élément dans une page HTML il faut toujours accéder à l'objet **Document** en premier.

<code>document.getElementById(id)</code>	Trouver un élément par son ID
<code>document.getElementsByTagName(name)</code>	Trouver des éléments à partir de l'attribut name
<code>document.getElementsByClassName(className)</code>	Trouver des éléments à partir d'une class

Ajouter ou supprimer des éléments HTML:

<code>document.createElement(elm)</code>	Créer un élément HTML
<code>document.removeChild(elm)</code>	Supprimer un élément HTML
<code>document.appendChild(elm)</code>	Ajouter un élément HTML
<code>document.replaceChild(new, old)</code>	Remplacer un élément HTML
<code>document.write(text)</code>	A essayer par vous même !!!!!

d. DOM css

L'HTML DOM permet à JavaScript de changer le style des éléments HTML.

```
document.getElementById(id).style.property = new style
```

e. DOM nodes

Pour ajouter un nouvel élément au DOM HTML, il faut créer l'élément puis l'ajouter à un élément existant.

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const element = document.getElementById("div1");
element.appendChild(para);
</script>
```

10. JS WEB API

a. Web API introduction

API: Application Programming Interface.

“En informatique, une interface de programmation d'application ou interface de programmation applicative souvent désignée par le terme API pour « Application Programming Interface » en anglais, est un ensemble normalisé de classes, de méthodes, de fonctions et de constantes qui sert de façade par laquelle un logiciel offre des services à d'autres logiciels.” Wikipedia.

Web API: est une API pour le web.

Tous les navigateurs ont un certain nombre de Web API qui peuvent supporter des opérations complexes et d'aider à l'accès aux données.

Par exemple, Geolocation API permet de retourner les coordonnées géographique du navigateur de l'utilisateur.

```
const myElement = document.getElementById("demo");

function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition);
  } else {
    myElement.innerHTML = "Geolocation is not supported by this browser.";
  }
}

function showPosition(position) {
  myElement.innerHTML = "Latitude: " + position.coords.latitude +
    "<br>Longitude: " + position.coords.longitude;
}
```

b. Web Storage API

Web storage API permet de stocker et de récupérer des données dans le navigateur.

```
localStorage.setItem("name", "John Doe");  
localStorage.getItem("name");
```

L'objet `localStorage` permet de stocker des données sur un site web particulier. Il est possible de stocker, lire, ajouter, modifier et de supprimer des données pour le site en question.

Les données seront stockées indéfiniment.

Voici un exemple de code qui permet d'ajouter la clé `name` avec une valeur Jhon Doe:

```
localStorage.setItem("name", "John Doe");
```

Dans le bout de code suivant on récupère via la clé `name`

```
localStorage.getItem("name");
```

L'objet `sessionStorage` permet de stocker des données pour une session c'est à dire que les données seront supprimées quand le navigateur sera fermé.

```
sessionStorage.setItem("name", "John Doe");
```

```
sessionStorage.getItem("name");
```


c. Web fetch API

C'est un outil moderne et puissant qui te permet de faire des requêtes réseau. Le plus souvent, il est utilisé pour envoyer ou recevoir des données d'un serveur, sans avoir à recharger toute la page web.

Exemple:

```
<script>
getText("fetch_info.txt");

async function getText(file) {
  let myObject = await fetch(file);
  let myText = await myObject.text();
  document.getElementById("demo").innerHTML = myText;
}
</script>
```

Dans l'exemple suivant on essaye de récupérer une liste de film sur une adresse web:

```
async function afficherFilms() {
  const reponse = await fetch("http://example.com/films.json");
  const films = await reponse.json();
  console.log(films);
}
```

La variable `reponse` ne contient pas encore la liste des films du coup on fait appel à la méthode `json()` qui renvoi un **PROMISES** dont la résolution fournit le résultat au format JSON.

Voici en image les options qu'on peut avoir avec la méthode `fetch`

JS Copy

```
// Exemple d'implémentation pour une requête POST
async function postData(url = "", donnees = {}) {
  // Les options par défaut sont indiquées par *
  const response = await fetch(url, {
    method: "POST", // *GET, POST, PUT, DELETE, etc.
    mode: "cors", // no-cors, *cors, same-origin
    cache: "no-cache", // *default, no-cache, reload, force-cache, only-if-
    cached
    credentials: "same-origin", // include, *same-origin, omit
    headers: {
      "Content-Type": "application/json",
      // 'Content-Type': 'application/x-www-form-urlencoded',
    },
    redirect: "follow", // manual, *follow, error
    referrerPolicy: "no-referrer", // no-referrer, *no-referrer-when-downgrade,
    origin, origin-when-cross-origin, same-origin, strict-origin, strict-origin-
    when-cross-origin, unsafe-url
    body: JSON.stringify(donnees), // le type utilisé pour le corps doit
    correspondre à l'en-tête "Content-Type"
  });
  return response.json(); // transforme la réponse JSON reçue en objet
  JavaScript natif
}

postData("https://example.com/solution", { solution: 42 }).then((donnees) => {
  console.log(donnees); // Les données JSON analysées par l'appel
  `donnees.json()`
});
```

[API Fetch documentation plus poussée](#)